# Machine Learning Tasks, Data Architecture, Learning Problems, and Evaluation

Prepared for Civil Engineering students beginning machine learning study

This reading introduces two linked topics. Subject 1 explains the main kinds of machine learning tasks and shows how civil engineering data is organized for learning. Subject 2 explains common learning problem types, practical evaluation metrics, and the data pitfalls that can quietly damage a project. The focus and examples are framed for real civil engineering work.

**Learning goals:** By the end of this reading, you should be able to identify the main ML task type in a civil engineering problem, describe one datapoint in tabular, image, or time-series form, explain the roles of training, validation, and test data, distinguish supervised, unsupervised, and semi-supervised learning, and choose evaluation metrics that match safety, cost, or reliability goals.

**Reference basis.** This tutorial follows standard machine learning terminology and evaluation guidance from the scikit-learn User Guide and Google's Machine Learning Crash Course, while the civil engineering examples are aligned with recent review literature on AI and ML applications in civil engineering [1]-[10].

# Acronyms & Parameters

ML = Machine Learning
AI = Artificial Intelligence
CCTV = Closed-Circuit Television
MAE = Mean Absolute Error
MSE = Mean Squared Error
RMSE = Root Mean Squared Error

$R^2$ = R-squared
F1 = F1 score
TP = True Positives
FP = False Positives
FN = False Negatives
X = input features / inputs
y = output / target / label

Y = output variables
mm = millimeters
MPa = megapascals
$m^3/s$ = cubic meters per second

# Subject 1. ML Tasks and Data Architecture

Machine learning is most useful when civil engineers have repeated examples, measurable variables, and a clear decision, forecast, or pattern to automate. At a high level, many applications can be grouped into four task families: prediction, recognition, feature extraction, and acceleration.

**Table 1. Core ML task types with civil engineering examples**

| Task family | What the model does | Civil engineering example | Typical output |
|---|---|---|---|
| Prediction / estimation | Predict a number or category from measured inputs | Estimate concrete compressive strength from mix variables; predict flood depth from rainfall and terrain; forecast traffic flow | One value or one class |
| Recognition | Identify whether a pattern is present and often where it is located | Detect cracks or potholes in images; identify damaged bridge components; flag pipe defects from CCTV frames | Class, box, or mask |
| Feature extraction | Turn raw measurements into a smaller and more informative representation | Extract damage-sensitive indicators from vibration signals; compress many sensor channels into a few state variables | New features |
| Acceleration / surrogate modeling | Replace a slow repeated computation with a faster approximation | Approximate many finite element runs; accelerate flood inundation maps; emulate repeated design studies | Fast estimate of a simulation or response |

The four task families often overlap. For example, a crack inspection system may first perform recognition on images, then extract features from the detected crack pattern, and finally predict deterioration severity or maintenance priority. Recent civil engineering reviews show broad use of these task types across structures, geotechnics, transportation, water systems, and construction management [10].

## 1.1 What is one datapoint?

A datapoint is one example given to the model. In **supervised learning**, one datapoint usually contains inputs X and output y. Inputs are also called features, variables, or attributes. The output is also called the target or label. Some problems have one output, while others have several. Figure 1 shows the basic idea for a tabular repository.

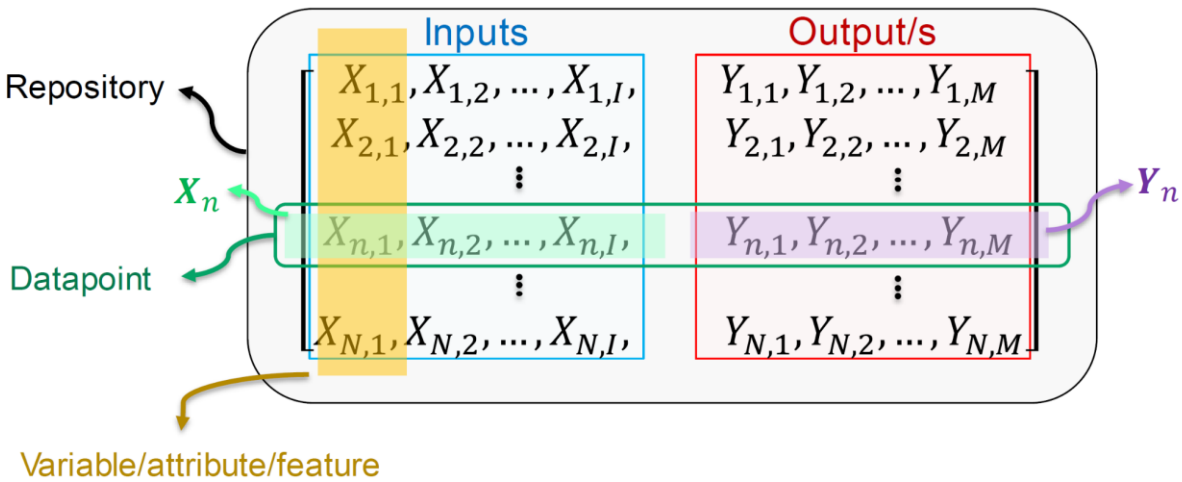



*Figure 1. Conceptual view of a repository, one datapoint, its input features X, and its output variables Y.*

For a concrete strength model, one datapoint could be one mix design and its tested strength. For a bridge image classifier, one datapoint could be one image and its crack label. For a traffic or vibration model, one datapoint could be one time window and the response linked to that window. The key idea is always the same: define one consistent unit of observation before any modeling starts. Note that having inputs is mandatory, but outputs are not, and it depends on the task at hand and the learning problem.

## 1.2 Common data formats in civil engineering

Civil engineering ML projects most often start from tabular data, image data, or time-series data. Each format can represent the same physical system in a different way, so the format should match the engineering question. Figure 2 summarizes the three common formats.

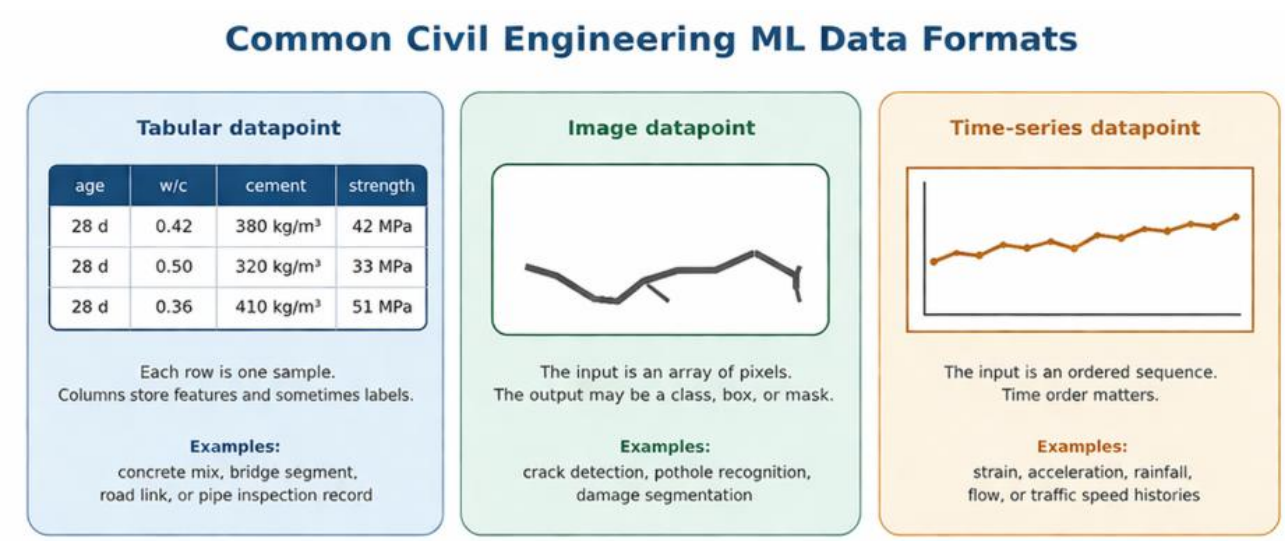



*Figure 2. Examples of one datapoint in tabular, image, and time-series form for civil engineering problems.*

**Table 2. How one datapoint changes across common data formats**

| Format | What one datapoint looks | Civil engineering example | When it fits well |
|---|---|---|---|

| | like | | |
|---|---|---|---|
| Tabular | Rows are samples; columns are features | Concrete mix variables, bridge inventory tables, pavement distress summaries | Best when each case can be summarized by measured variables |
| Image | Pixels form the input array | Drone images of cracks, tunnel lining photos, and road surface images | Best when spatial appearance matters |
| Time-series | Ordered sequence over time | Acceleration records, strain histories, rainfall-flow series, and traffic speed | Best when timing and sequence matter |

**Practical tip:** Before building any model, write one sentence that defines the datapoint. For example: 'One datapoint is one bridge deck image,' or 'One datapoint is one 10-minute sensor window.' This simple step prevents many later mistakes. Prediction problems are the easiest place to start because the question is clear: given inputs, what output should we estimate? Recognition and feature extraction are common when the data are images or sensor signals. Acceleration becomes important when a physical simulation is accurate but too slow to run many times.

## 1.3 Training, validation, and test data

Training data is used to fit model parameters. Validation data is used during development to compare settings, features, or model choices. Test data is kept aside for the final check on unseen examples [4,8]. Testing on the same data used for training can make a weak model look excellent, which is why a clean split is essential [4]. Think of an ML model as a student: she studies the training materials and then is tested on the exam with test data. This student has parameters, constraints, and limitations. Learning is about gradually finding the optimal parameters so that the task is performed well, becoming comfortable with the training materials, and achieving a good score on the exam using the test data.

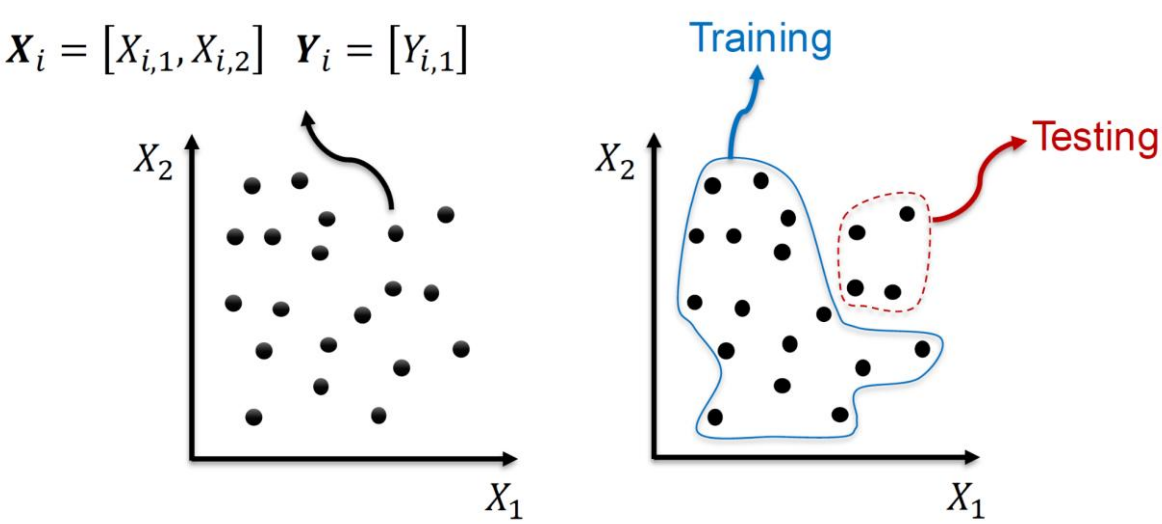



*Figure 3. One example of separating datapoints into training and testing subsets in the same feature space.*

A three-way split is often more useful than a simple train-test split because it keeps the test set for the final report and uses the validation set for repeated tuning [8]. Validation is like our student setting aside a part of the training materials and using them to keep validating himself throughout the semester, leading up to exam day. There is no single mandatory percentage, but training is usually the largest subset. The key requirement is that validation and test examples should represent the kind of data the model will face later [8].

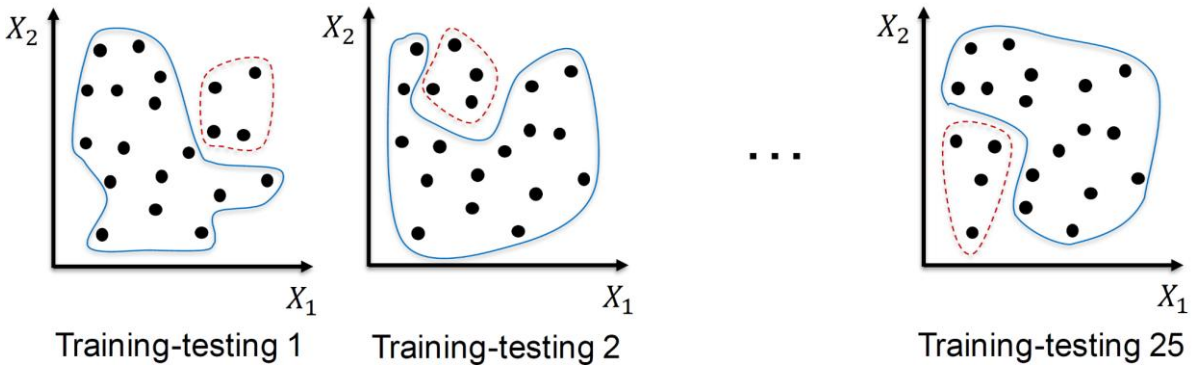



*Figure 4. Repeated random train-test divisions illustrate why one lucky split should not be trusted by itself.*

Figure 4 also shows why a single score from a single split is not sufficient when the data is limited. Repeated random splits or cross-validation help estimate whether the model is consistently strong or only benefited from an easy split [4]. Imagine if our student (ML model) is given easy training materials and encounters a hard test on exam day, or hard training data and an easy test on exam day; in both cases, it's unfair. Instead, imagine students are repeatedly trained and examined, each time using a different train-test set; the average student's training and testing performance would be a good representation of the student's capability. This matters in civil engineering because datasets are often small, expensive to label, and collected under narrow field conditions.

**Civil engineering splitting rule:** Split by the real deployment unit whenever possible. If the model will be used on new bridges, split by bridge. If it will be used on future storms, split by event or by time. If records from the same bridge, site, or day appear in both training and test sets, the evaluation can look better than true field performance.

# Subject 2. Learning Problems and Evaluation

The first question in any ML project is not 'Which algorithm should I use?' but rather 'What kind of learning problem do I have?' Problem type determines the data you need, the output you expect, and the metric that should guide decisions.

## 2.1 Supervised, unsupervised, and semi-supervised learning

Supervised learning uses labeled outputs. scikit-learn groups regression and classification under supervised learning [1]. Unsupervised learning works with inputs only and looks for structure such as clusters or lower-dimensional representations [2]. Semi-supervised learning sits between them: only some training examples are labeled, which is useful when labeling is expensive but raw data is plentiful [3].

**Table 3. Common learning problem types and their evaluation style**

| Problem type | Data structure | Civil engineering example | Typical evaluation |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Supervised regression | Inputs X and numeric output y | Predict bridge deflection, flood depth, compressive strength, settlement | MAE, MSE, RMSE, $R^2$ |
| Supervised classification | Inputs X and categorical y | Crack / no-crack, safe / warning / danger, soil class | Accuracy, precision, recall, F1, confusion matrix |
| Unsupervised clustering | Inputs only | Group similar traffic patterns, vibration states, or zones with similar geotechnical behavior | Silhouette score, stability, engineering usefulness |
| Unsupervised dimensionality reduction | Inputs only | Compress many sensor channels into a few indicators or visual axes | Explained variance, reconstruction error, downstream usefulness |
| Generative problems | Usually inputs only, sometimes conditional labels | Generate realistic crack images, synthetic sensor traces, or design candidates | Sample quality, reconstruction error, expert review, downstream value |
| Semi-supervised learning | Small labeled set plus large unlabeled set | Few labeled defect images and many unlabeled site images | Use the metric of the final supervised task |

## 2.2 Regression and classification

Regression predicts a number. Examples include predicting compressive strength, travel time, scour depth, bearing capacity, or pipe flow. Classification predicts a category. Examples include whether a bridge component is damaged, which pavement distress type is present, or whether a slope is low, medium, or high risk. These are the two most common starting points for beginner civil engineering projects [1,10].

A useful habit is to write the output in plain language before selecting the model. If the output sentence starts with 'how much' or 'how many,' regression is likely. If it starts with 'which class' or 'which category,' classification is likely.

## 2.3 Unsupervised and semi-supervised learning

Unsupervised learning is valuable when no agreed-upon labels are available yet. Clustering can reveal groups of bridges, traffic states, or soil behaviors that look similar. Dimensionality reduction can compress many measured variables into a smaller set that is easier to visualize or model [2]. Generative models go one step further and try to learn how the data itself is distributed so they can produce realistic new samples or reconstructions.

Semi-supervised learning is often practical in civil engineering because expert labels are expensive. For example, an engineer may have labels for only a small number of defect images or corrosion states, while thousands of unlabeled images are available. Semi-supervised methods try to use both types of data together [3].

**Beginner rule:** If you have reliable labels and a clear target, start with supervised learning. If labels are scarce or

unclear, use unsupervised exploration first and treat the results as engineering clues, not final truth.

## 2.4 Core evaluation metrics

**Table 4. Beginner-friendly metric cheat sheet**

| Metric | Simple formula or idea | What it tells you | When it is especially useful |
|---|---|---|---|
| MAE | Average absolute error | Average miss measured in the original unit | Easy to interpret when units matter, such as mm, MPa, or $m^3/s$ |
| MSE | Average squared error | Penalizes large mistakes strongly | Useful when large misses are especially costly |
| RMSE | Square root of MSE | Large errors still matter more, but the result returns to the original unit | Common for prediction reports |
| Accuracy | Correct predictions / all predictions | Overall fraction correct | Only reliable when classes are reasonably balanced |
| Precision | TP / (TP + FP) | Of all positive alerts, how many were truly positive? | Best when false alarms are costly |
| Recall | TP / (TP + FN) | Of all true positives, how many were found? | Best when missed detections are dangerous |
| F1 score | 2PR / (P + R) | Balances precision and recall | Useful for imbalanced classification when both types of error matter |
| Silhouette score | Cluster separation measure | Higher values mean points fit their own cluster better than nearby clusters | Useful for clustering, but still needs engineering interpretation |

Metric choice should follow the final engineering decision, not just textbook habit. scikit-learn's evaluation guide emphasizes starting from the application goal and the decision that will be made from the prediction [5]. Google's ML Crash Course similarly notes that accuracy alone is a poor guide on imbalanced problems, and that precision, recall, and F1 often give better insight [7].

## 2.5 Choosing metrics for safety, cost, and reliability

**Table 5. Match the metric to the engineering consequence**

| Application | Main concern | Metrics to prioritize | Why |
|---|---|---|---|
| Bridge damage screening | Missing real damage | Recall, then F1 | A false negative can delay inspection of a true defect |
| Automatic defect alerts for maintenance crews | Too many false alarms | Precision | Low precision can waste field time and budget |

| | | | |
|---|---|---|---|
| Flood depth prediction | Large numeric misses | MAE and RMSE | Report both average miss and sensitivity to large misses |
| Traffic state classification | Confusing rare congestion states | Per-class recall and confusion matrix | Overall accuracy can hide poor performance on rare but important classes |
| Material strength prediction | Outliers that are unsafe for design | MAE, RMSE, error plots near limits | Look beyond one average score |
| Condition rating models for assets | Uneven performance across asset groups | Per-group metrics and calibration checks | A fair average may still hide weak performance on one subgroup |

In short, recall matters when missing a positive case is dangerous, precision matters when false alarms are expensive, and F1 is a useful compromise when both matter [7]. For regression, MAE is often the easiest to explain, while MSE or RMSE become more important when large misses are unacceptable. For unsupervised tasks, no single number should replace engineering judgment.

## 2.6 Data pitfalls in civil engineering ML

**Table 6. Common data pitfalls and how to spot them**

| Pitfall | What it means | Civil engineering example | Basic safeguard |
|---|---|---|---|
| Sampling bias | Training data comes from a narrow or unrepresentative sample | Only urban bridges, only daytime traffic, only dry-season imagery | Collect broader data or state limits clearly [9] |
| Distribution shift | Deployment data differs from development data | New camera angle, different sensor brand, extreme weather year, new site geology | Test on truly future or out-of-site data [8] |
| Data leakage | Information from the future or the test set sneaks into training | Using the same bridge in both sets, using future measurements to predict past failure | Split by asset, site, or time before feature engineering [4,8] |
| Inconsistent preprocessing | Transformations are not applied the same way to all sets | Scaling training data but forgetting to scale test data | Use one repeatable preprocessing pipeline [6] |
| Overfitting | Model memorizes training details and fails to generalize | Excellent training score but unstable field performance | Use validation, cross-validation, simpler baselines, and error inspection [4] |

Sampling bias occurs when the collected data is not representative [9]. In civil engineering, that might mean a bridge dataset dominated by one climate region or one bridge type. Distribution shift occurs when the model is deployed on data that no longer looks like the development data, such as after a sensor replacement, a change in lighting, or an extreme weather event [8]. These two issues are especially important in safety-critical and site-specific applications [10].

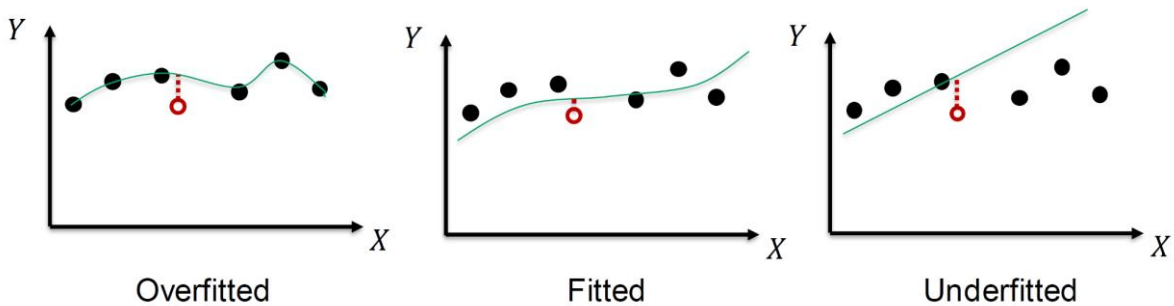



*Figure 5. Overfitted, appropriately fitted, and underfitted models. A good model should generalize beyond the training points.*

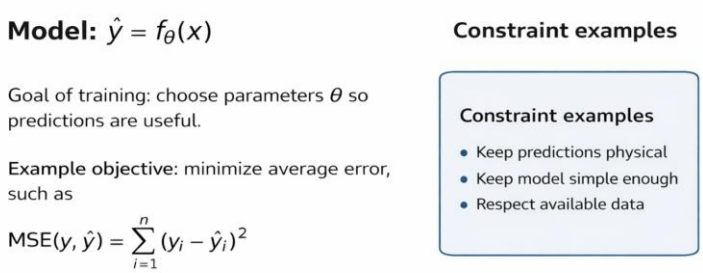
Figure 5 gives the intuition behind overfitting and underfitting. An overfitted model follows the training data too closely and can fail on new data. An underfitted model is too simple to capture useful structure. The goal is not maximum complexity, but the best generalization to unseen civil engineering cases [4].

**Notebook-ready starter workflow:** 1) Define the engineering question and the unit of prediction. 2) Build one clean datapoint definition. 3) Split the data by site, asset, or time. 4) Train a simple baseline first. 5) Choose a metric that matches the cost of errors. 6) Inspect the worst predictions with engineering judgment. 7) Only then try more advanced models.

## Machine Learning Mathematics

**Machine learning** can be thought of as building a function that turns inputs into useful outputs. In symbols, we often write this as $\hat{y} = f_\theta(x)$ where $x$ is the input, $\hat{y}$is the predicted output, and $\theta$ stands for the model parameters that must be learned from data. During training, the algorithm adjusts those parameters to minimize an objective function, which measures how bad the current predictions are. Lower is better.

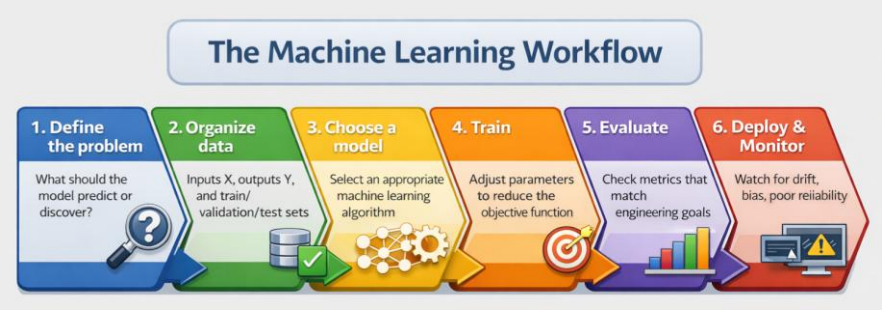
**For beginners, the safest mental model is this:** machine learning is a structured way to learn patterns from examples. In civil engineering, those examples might be bridge sensor readings, concrete mix properties, pavement images, traffic counts, or river level histories.




In a Machine learning workflow, first, you define the problem by deciding exactly what you want the model to predict, such as concrete strength, traffic flow, bridge condition, or slope stability. Then you organize the data by collecting the input variables and the correct output values, cleaning errors, and splitting the data into training, validation, and test sets so the model can learn and then be checked fairly.

Next, you choose a model and train it. Choosing a model means selecting a suitable method, such as regression, decision trees, or neural networks, depending on the problem and the data (we will go over such models in the next Modules). Training means the model learns patterns from past examples by adjusting itself to reduce

prediction errors. For civil engineering students, this is similar to calibrating an analysis model so that it matches measured behavior more closely.




Finally, you evaluate the model and then deploy and monitor it. Evaluation checks whether the model is accurate, reliable, and useful for engineering decisions, especially on new data it has not seen before. Deployment means using the model in practice, but monitoring is still necessary because conditions can change over time, data quality can worsen, and the model may become less reliable. Machine learning is not just about running an algorithm, but about following a careful engineering process from problem definition to long-term use.

### Key takeaways

- Machine learning tasks in civil engineering often fall into prediction, recognition, feature extraction, or acceleration.
- A datapoint is one consistent unit of observation. Its structure changes across tabular, image, and time-series data.
- Training, validation, and test sets serve different roles. Keep the test set for the final check.
- Regression predicts numbers, classification predicts categories, unsupervised learning finds structure without labels, and semi-supervised learning uses both labeled and unlabeled data.
- Choose metrics by engineering consequence. Accuracy alone is rarely enough.
- Watch for sampling bias, distribution shift, leakage, inconsistent preprocessing, and overfitting.

## References

[1] scikit-learn developers, “1. Supervised learning,” scikit-learn User Guide, version 1.8.0, 2026. https://scikit-learn.org/stable/supervised_learning.html

[2] scikit-learn developers, “2. Unsupervised learning,” scikit-learn User Guide, version 1.8.0, 2026. https://scikit-learn.org/stable/unsupervised_learning.html

[3] scikit-learn developers, “1.14. Semi-supervised learning,” scikit-learn User Guide, version 1.8.0, 2026. https://scikit-learn.org/stable/modules/semi_supervised.html

[4] scikit-learn developers, “3.1. Cross-validation: evaluating estimator performance,” scikit-learn User Guide, version 1.8.0, 2026. https://scikit-learn.org/stable/modules/cross_validation.html

[5] scikit-learn developers, “3.4. Metrics and scoring: quantifying the quality of predictions,” scikit-learn User Guide, version 1.8.0, 2026. https://scikit-learn.org/stable/modules/model_evaluation.html

[6] scikit-learn developers, “11. Common pitfalls and recommended practices,” scikit-learn User Guide, version 1.8.0, 2026. https://scikit-learn.org/stable/common_pitfalls.html

[7] Google for Developers, “Classification: Accuracy, precision, recall, and related metrics,” Machine Learning Crash Course, Jan. 12, 2026. https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall

[8] Google for Developers, “Datasets: Dividing the original dataset,” Machine Learning Crash Course, Dec. 3, 2025. https://developers.google.com/machine-learning/crash-course/overfitting/dividing-datasets

[9] Google for Developers, “Fairness: Types of bias,” Machine Learning Crash Course, Dec. 3, 2025. https://developers.google.com/machine-learning/crash-course/fairness/types-of-bias

[10] A. Bahadori-Jahromi, S. Room, C. Paknahad, M. Altekreeti, Z. Tariq, and H. Tahayori, “The Role of Artificial Intelligence and Machine Learning in Advancing Civil Engineering: A Comprehensive Review,” Applied Sciences, vol. 15, no. 19, art. 10499, 2025. https://doi.org/10.3390/app151910499